

Python Capstone Project Solution

Basic Web Vulnerability Scanner

Based on the requirements from the uploaded PDF. [filecite turn0file0 L1-L1](#)

1. Project Breakdown

Project Title

Basic Web Vulnerability Scanner using Python

Main Goal

The purpose of this project is to build a simple cybersecurity tool that scans a website for:

- SQL Injection (SQLi)
- Cross-Site Scripting (XSS)

The scanner works by:

1. Accepting a website URL from the user
 2. Loading the webpage
 3. Detecting HTML forms
 4. Sending normal and malicious test inputs
 5. Checking the website response
 6. Reporting possible vulnerabilities
-

2. Technologies Used

Technology	Purpose
Python	Main programming language
requests	Sending HTTP requests
BeautifulSoup	Extracting HTML forms
urllib.parse	Handling URLs
re	Pattern matching for SQL errors

3. Project Folder Structure

```
capstone_project/  
|  
├─ scanner.py  
├─ requirements.txt  
├─ README.md  
└─ scan_report.txt
```

4. Installation Guide

Step 1: Install Python

Download Python from:

<https://www.python.org>

Step 2: Install Required Libraries

Run this command in the terminal:

```
pip install requests beautifulsoup4
```

5. Complete Python Source Code (scanner.py)

```
# =====  
# BASIC WEB VULNERABILITY SCANNER  
# CAPSTONE PROJECT  
# =====  
  
# Import requests module for sending HTTP requests  
import requests  
  
# Import BeautifulSoup for HTML parsing  
from bs4 import BeautifulSoup
```

```

# Import urljoin to combine base URLs and form actions
from urllib.parse import urljoin

# Import regular expressions module
import re

# =====
# EDUCATIONAL WARNING MESSAGE
# =====

print("=" * 60)
print("WARNING: This tool is for educational purposes only")
print("Only scan websites you own or have permission to test")
print("=" * 60)

# =====
# FUNCTION TO GET ALL FORMS FROM A WEBPAGE
# =====

# This function accepts a URL and returns all forms found on the page
# parameter:
# url -> target website address

def get_forms(url):

    # Send GET request to website
    response = requests.get(url)

    # Parse HTML content using BeautifulSoup
    soup = BeautifulSoup(response.content, "html.parser")

    # Find and return all form tags
    return soup.find_all("form")

# =====
# FUNCTION TO EXTRACT FORM DETAILS
# =====

# This function extracts:
# - form action
# - form method
# - all input fields

def get_form_details(form):

    # Create dictionary to store form information
    details = {}

```

```

# Get form action attribute
action = form.attrs.get("action")

# Get form method (GET or POST)
method = form.attrs.get("method", "get").lower()

# Create empty list for inputs
inputs = []

# Find all input tags inside the form
for input_tag in form.find_all("input"):

    # Get input type
    input_type = input_tag.attrs.get("type", "text")

    # Get input name
    input_name = input_tag.attrs.get("name")

    # Store input information in dictionary
    inputs.append({
        "type": input_type,
        "name": input_name
    })

# Store all extracted information
details["action"] = action
details["method"] = method
details["inputs"] = inputs

# Return complete form details
return details

# =====
# FUNCTION TO SUBMIT FORMS
# =====

# This function sends payloads to the form
# parameters:
# form_details -> extracted form information
# url -> target website
# value -> payload or test value

def submit_form(form_details, url, value):

    # Combine base URL and form action
    target_url = urljoin(url, form_details["action"])

```

```

# Create dictionary to store data to send
data = {}

# Loop through all inputs
for input_field in form_details["inputs"]:

    # Get input name
    input_name = input_field.get("name")

    # Only send data if input has a name
    if input_name:

        # Insert payload into field
        data[input_name] = value

# Check if form method is POST
if form_details["method"] == "post":

    # Send POST request
    return requests.post(target_url, data=data)

else:

    # Send GET request
    return requests.get(target_url, params=data)

# =====
# FUNCTION TO TEST SQL INJECTION
# =====

# This function tests a webpage for SQL Injection vulnerabilities

def scan_sql_injection(url):

    # SQL injection payloads
    sql_payloads = [
        "' OR '1'='1",
        "admin' --",
        "test123"
    ]

    # Common SQL database error messages
    sql_errors = {
        "you have an error in your sql syntax",
        "warning: mysql",
        "unclosed quotation mark",
        "quoted string not properly terminated",
        "sql syntax",
    }

```

```

        "mysql_fetch",
        "sqlite error",
        "oracle error"
    }

    # Get all forms from target page
    forms = get_forms(url)

    # Create list to store vulnerabilities
    vulnerabilities = []

    # Loop through all forms
    for form in forms:

        # Extract form details
        form_details = get_form_details(form)

        # Test each SQL payload
        for payload in sql_payloads:

            # Submit form with payload
            response = submit_form(form_details, url, payload)

            # Convert response text to lowercase
            content = response.text.lower()

            # Check for SQL error messages
            for error in sql_errors:

                # If SQL error is detected
                if error in content:

                    # Store vulnerability information
                    vulnerabilities.append({
                        "type": "SQL Injection",
                        "payload": payload,
                        "form_action": form_details["action"]
                    })

                    # Stop checking once vulnerability is found
                    break

    # Return all SQL vulnerabilities found
    return vulnerabilities

# =====
# FUNCTION TO TEST XSS VULNERABILITIES
# =====

```

```

# This function checks if script payloads are reflected back

def scan_xss(url):

    # XSS payload
    xss_payload = "<script>alert(1)</script>"

    # Get all forms on the page
    forms = get_forms(url)

    # Create list to store vulnerabilities
    vulnerabilities = []

    # Loop through each form
    for form in forms:

        # Extract form details
        form_details = get_form_details(form)

        # Submit XSS payload
        response = submit_form(form_details, url, xss_payload)

        # Check if payload appears in response
        if xss_payload in response.text:

            # Store vulnerability information
            vulnerabilities.append({
                "type": "Cross-Site Scripting (XSS)",
                "payload": xss_payload,
                "form_action": form_details["action"]
            })

    # Return vulnerabilities found
    return vulnerabilities

# =====
# MAIN PROGRAM
# =====

# Ask user to enter target URL
url = input("Enter website URL to scan: ")

# Try connecting to the website
try:

    # Send request to validate URL accessibility
    response = requests.get(url)

```

```

# Check if request was successful
if response.status_code == 200:

    print("\nWebsite is accessible")

else:

    print("\nWebsite returned status code:", response.status_code)

# Handle connection errors
except requests.exceptions.RequestException:

    print("\nUnable to access website")

    # Stop program execution
    exit()

# =====
# DETECT FORMS
# =====

# Get all forms from website
forms = get_forms(url)

# Display number of forms found
print(f"\nDetected {len(forms)} forms on the website")

# =====
# START SQL INJECTION SCAN
# =====

print("\nStarting SQL Injection scan...")

# Run SQL Injection scanner
sql_vulnerabilities = scan_sql_injection(url)

# =====
# START XSS SCAN
# =====

print("\nStarting XSS scan...")

# Run XSS scanner
xss_vulnerabilities = scan_xss(url)

# =====
# FINAL REPORT

```

```

# =====

print("\n" + "=" * 60)
print("SCAN REPORT")
print("=" * 60)

# Display target URL
print(f"Target URL: {url}")

# Display number of forms detected
print(f"Forms Detected: {len(forms)}")

# Display SQL Injection results
print("\nSQL Injection Results:")

# Check if SQL vulnerabilities were found
if sql_vulnerabilities:

    # Display each vulnerability
    for vulnerability in sql_vulnerabilities:

        print("- Possible SQL Injection Detected")
        print("  Form Action:", vulnerability["form_action"])
        print("  Payload:", vulnerability["payload"])

else:

    print("No SQL Injection vulnerabilities detected")

# Display XSS results
print("\nXSS Results:")

# Check if XSS vulnerabilities were found
if xss_vulnerabilities:

    # Display each vulnerability
    for vulnerability in xss_vulnerabilities:

        print("- Possible XSS Vulnerability Detected")
        print("  Form Action:", vulnerability["form_action"])
        print("  Payload:", vulnerability["payload"])

else:

    print("No XSS vulnerabilities detected")

# =====
# SAVE REPORT TO FILE

```

```

# =====

# Open text file in write mode
with open("scan_report.txt", "w") as file:

    # Write report header
    file.write("WEB VULNERABILITY SCANNER REPORT\n")

    # Write target URL
    file.write(f"Target URL: {url}\n")

    # Write forms detected
    file.write(f"Forms Detected: {len(forms)}\n\n")

    # Write SQL results
    file.write("SQL Injection Results:\n")

    # Check SQL vulnerabilities
    if sql_vulnerabilities:

        # Write each SQL vulnerability
        for vulnerability in sql_vulnerabilities:

            file.write("Possible SQL Injection Detected\n")
            file.write(f"Form Action: {vulnerability['form_action']}\n")
            file.write(f"Payload: {vulnerability['payload']}\n\n")

    else:

        file.write("No SQL Injection vulnerabilities detected\n\n")

    # Write XSS results
    file.write("XSS Results:\n")

    # Check XSS vulnerabilities
    if xss_vulnerabilities:

        # Write each XSS vulnerability
        for vulnerability in xss_vulnerabilities:

            file.write("Possible XSS Vulnerability Detected\n")
            file.write(f"Form Action: {vulnerability['form_action']}\n")
            file.write(f"Payload: {vulnerability['payload']}\n\n")

    else:

        file.write("No XSS vulnerabilities detected\n")

```

```
# Display completion message
print("\nReport saved to scan_report.txt")

# =====
# END OF PROGRAM
# =====
```

6. requirements.txt

```
requests
beautifulsoup4
```

7. README.md

```
# Basic Web Vulnerability Scanner

## Description
This project is a simple Python-based web vulnerability scanner.
It checks websites for common security vulnerabilities such as:

- SQL Injection (SQLi)
- Cross-Site Scripting (XSS)

The program scans HTML forms and tests them using malicious payloads.

---

## Features

- Accepts a website URL
- Detects HTML forms automatically
- Extracts form details
- Tests SQL Injection payloads
- Tests XSS payloads
- Displays scan report in terminal
- Saves report to a text file

---

## Technologies Used
```

```
- Python
- requests
- BeautifulSoup

---

## Installation

Install required libraries:

```bash

pip install requests beautifulsoup4
```

---

## How to Run

Run the scanner:

```
python scanner.py
```

Then enter the website URL.

Example:

```
Enter website URL to scan: http://example.com
```

---

## Example Output

```
=====
WARNING: This tool is for educational purposes only
=====

Enter website URL to scan: http://testphp.vulnweb.com

Website is accessible
Detected 2 forms on the website

Starting SQL Injection scan...
Starting XSS scan...
```

```
=====
SCAN REPORT
=====
Target URL: http://testphp.vulnweb.com
Forms Detected: 2

SQL Injection Results:
No SQL Injection vulnerabilities detected

XSS Results:
Possible XSS Vulnerability Detected
```

---

## Ethical Warning

This tool is for educational purposes only. Only scan websites you own or have permission to test.

---

## Author

Capstone Project Submission

```

8. How the Project Works

Step-by-Step Flow

Step 1 – User Enters URL
The user enters the target website URL.

Example:

```python
url = input("Enter website URL to scan: ")
```

Step 2 — Website Validation

The program checks whether the website is accessible.

```
requests.get(url)
```

Step 3 — Form Detection

BeautifulSoup searches for all HTML forms.

```
soup.find_all("form")
```

Step 4 — Extract Form Information

The scanner extracts:

- Form action
 - Form method
 - Input fields
-

Step 5 — Send Test Payloads

The scanner sends:

Normal Input

```
test123
```

SQL Injection Payloads

```
' OR '1'='1  
admin' --
```

XSS Payload

```
<script>alert(1)</script>
```

Step 6 — Analyze Responses

The scanner checks:

For SQL Injection

- SQL errors
- Database messages
- Unusual responses

For XSS

- Reflected script content
 - Missing sanitization
-

Step 7 — Generate Report

The program prints:

- Number of forms
- Vulnerabilities found
- Payloads used
- Form actions

It also saves results to:

```
scan_report.txt
```

9. Sample Viva Questions and Answers

Q1. What is SQL Injection?

SQL Injection is a web vulnerability where attackers insert malicious SQL code into input fields to manipulate a database.

Q2. What is XSS?

Cross-Site Scripting (XSS) is a vulnerability where attackers inject malicious JavaScript into webpages.

Q3. Why is BeautifulSoup used?

BeautifulSoup is used for parsing HTML and extracting forms from webpages.

Q4. Why do we use requests?

The requests library is used to send HTTP requests to websites.

Q5. Why is ethical warning important?

It reminds users to only scan websites they own or have permission to test.

10. Key Learning Outcomes

After completing this project, you understand:

- Web form handling
 - HTTP requests
 - HTML parsing
 - Basic cybersecurity testing
 - SQL Injection concepts
 - XSS concepts
 - Report generation
 - Ethical hacking principles
-

11. Final Notes

This project satisfies the Capstone requirements:

✓ Website input ✓ Form detection ✓ SQL Injection testing ✓ XSS testing ✓ Reporting system ✓ Ethical warning ✓ Python implementation ✓ Commented code ✓ README documentation

12. Possible Future Improvements

You can improve the project by adding:

- GUI using Tkinter
- Multi-threaded scanning
- CSRF detection
- Login authentication
- Colorful terminal output
- JSON report export
- Command-line arguments

- Better payload detection
 - Advanced vulnerability checks
-

END OF CAPSTONE PROJECT SOLUTION